

Algorytmy i złożoność obliczeniowa - Projekt

Sprawozdanie

Zadanie projektowe nr 1

Analiza algorytmów sortowania i wpływu struktury danych na czas wykonania

Mykhailo Tsonyev nr albumu 284497

Prowadzący kurs:

Termin zajęć: TN wtorek 9:15

Termin oddania zadania: 12 maja 2026

1. Wstęp

Celem projektu było zaimplementowanie programu sortującego dane rosnąco oraz przeprowadzenie analizy czasu działania wybranych algorytmów sortowania.

W zakresie wymaganym na ocenę 3.0 zaimplementowano algorytmy `CocktailSort`, `MergeSort` i `InsertionSort` oraz dwie własne struktury liniowe: `DynamicArray` i `SinglyLinkedList`. Program oprócz samego sortowania sprawdza poprawność wyniku i zapisuje wyniki pomiarów do pliku CSV.

Część eksperymentalna obejmuje trzy części: **Badanie A**, **Badanie B** i **Badanie C**. Ich szczegółowy układ oraz sposób interpretacji wyników zostały opisane w rozdziale **Metodyka badań**.

Program uruchamiano na komputerze MacBook Air M1 z 8 GB RAM. Dane eksperymentalne pochodzą z pliku `results/research_data.csv`.

2. Metodyka badań

Eksperyment został podzielony na trzy części: **Badanie A**, **Badanie B** i **Badanie C**. Każdy benchmark wykonano 50 razy.

W pliku CSV zapisano surowy czas każdej iteracji. Wartości `min_us`, `avg_us` i `max_us` pojawiają się tylko w ostatnim wierszu danego bloku 50 iteracji.

Rozkład benchmarków pomiędzy główne badania był następujący:

- **Badanie A** obejmuje 24 benchmarki: każda kombinacja `CocktailSort`, `MergeSort`, `InsertionSort` z `DynamicArray` i `SinglyLinkedList` dla rozmiarów 5000, 10000, 25000 i 50000.
- **Badanie B** obejmuje 8 benchmarków: `MergeSort` dla `DynamicArray` i `SinglyLinkedList`, a dla każdej struktury rozkłady `random`, `ascending`, `ascending50Per` i `descending`.
- **Badanie C** obejmuje 3 benchmarki: `MergeSort` dla `DynamicArray` i rozmiaru 25000 przy typach `int`, `float` i `unsigned int`.

3. Opis implementacji

Implementacja używa szablonów, dzięki czemu te same algorytmy mogą pracować na różnych typach danych bez powielania kodu. Wspólny interfejs struktur zapewnia klasa `IStructure<T>`, która udostępnia metody `size()`, `operator[]`, `pushBack()` oraz `swap()`. Taki układ pozwala stosować te same algorytmy dla tablicy i listy tam, gdzie jest to uzasadnione, a tam, gdzie koszt dostępu indeksowego byłby zbyt duży, przygotować osobną implementację dla `SinglyLinkedList` [2][5].

3.1. Struktura `DynamicArray`

`DynamicArray<T>` przechowuje dane w ręcznie zarządzanym buforze pamięci `data`, a dodatkowo zapisuje `currentSize` oraz `capacity`. Struktura udostępnia dostęp indeksowy przez `operator[]`, zwraca bieżącą liczbę elementów przez `size()`, pozwala dopisać element na koniec przez `pushBack()` i zamieniać miejscami dwa elementy przez `swap(i, j)`. Dodatkowo posiada konstruktor kopiujący, operator przypisania oraz destruktory, które odpowiadają za poprawne zarządzanie pamięcią.

Z punktu widzenia algorytmów najważniejsze jest to, że `DynamicArray` oferuje szybki dostęp do elementu o zadanym indeksie. Dzięki temu klasyczne wersje `CocktailSort`, `MergeSort` i `InsertionSort` są szybkie.

3.2. Struktura `SinglyLinkedList`

`SinglyLinkedList<T>` przechowuje elementy w węzłach `Node`, z których każdy zawiera wartość `value` i wskaźnik `next` na kolejny węzeł. Struktura posiada wskaźniki `head` i `tail`, dzięki czemu zna początek i koniec listy, a także pole `currentSize`, które przechowuje liczbę elementów. Publiczne

operacje obejmują `pushBack()`, `size()`, `operator[]` oraz `swap(i, j)`. W części prywatnej znajdują się też metody pomocnicze `clear()`, `copyFrom()` i `nodeAt(index)`.

W praktyce ta struktura jest używana inaczej niż `DynamicArray`. Dla listy nie opłaca się opierać sortowania na ciągłym korzystaniu z `operator[]`, dlatego dla części algorytmów przygotowano osobne implementacje. `CocktailSort` dla `SinglyLinkedList` najpierw tworzy tablicę wskaźników do węzłów, `MergeSort` dzieli i scala listę przez przepinanie `next`, a `InsertionSort` buduje nową uporządkowaną część listy przez wstawianie węzłów we właściwe miejsce. Dzięki temu algorytmy pracują zgodnie z naturą listy jednokierunkowej, zamiast udawać tablicę [2][5].

3.3. Algorytm CocktailSort

`CocktailSort` wykonuje dwa przejścia w obrębie aktualnego zakresu danych: najpierw od lewej do prawej, przesuwając większe elementy ku końcowi, a następnie od prawej do lewej, przesuwając mniejsze elementy ku początkowi. Dla `DynamicArray` algorytm korzysta bezpośrednio z `operator[]` i `swap()`.

Dla `SinglyLinkedList` przygotowano osobną implementację. Zamiast wielokrotnie wywoływać `nodeAt(index)` przez `operator[]`, algorytm najpierw buduje tymczasową tablicę wskaźników do kolejnych węzłów listy. Następnie porównuje i zamienia wartości węzłów, przechodząc po tej tablicy wskaźników. Gdyby wykorzystać listę wyłącznie przez `operator[]`, pojedynczy dostęp miałby koszt liniowy, a całe sortowanie otrzymałoby bardzo duży dodatkowy narzut, praktycznie zbliżający się do zachowania sześciennie rosnącego względem rozmiaru wejścia. Osobna implementacja ogranicza ten problem i zachowuje naturalny dla `CocktailSort` charakter kwadratowy [3][5].

3.4. Algorytm MergeSort

`MergeSort` działa według schematu dziel i zwyciężaj. Dla `DynamicArray` algorytm rekurencyjnie dzieli zakres indeksów na dwie połowy, sortuje każdą z nich, a następnie scala obie uporządkowane części. W czasie scalania używany jest jeden dodatkowy bufor pomocniczy, czyli tymczasowa tablica `buffer`, do której przepisywane są elementy w trakcie łączenia dwóch posortowanych fragmentów. Bufor ten jest alokowany tylko raz dla całego sortowania, a nie przy każdym poziomie rekurencji [1][5].

Dla `SinglyLinkedList` zastosowano osobną implementację opartą na wskaźnikach. Lista jest dzielona na dwie połowy metodą szybkiego i wolnego wskaźnika, a następnie scalana przez przepinanie pól `next`, bez kopiowania całej listy do osobnej struktury i bez używania `operator[]`. Taki wariant lepiej odpowiada właściwościom listy jednokierunkowej i pozwala utrzymać oczekiwaną złożoność $O(n \log n)$ [1][2][5].

3.5. Algorytm InsertionSort

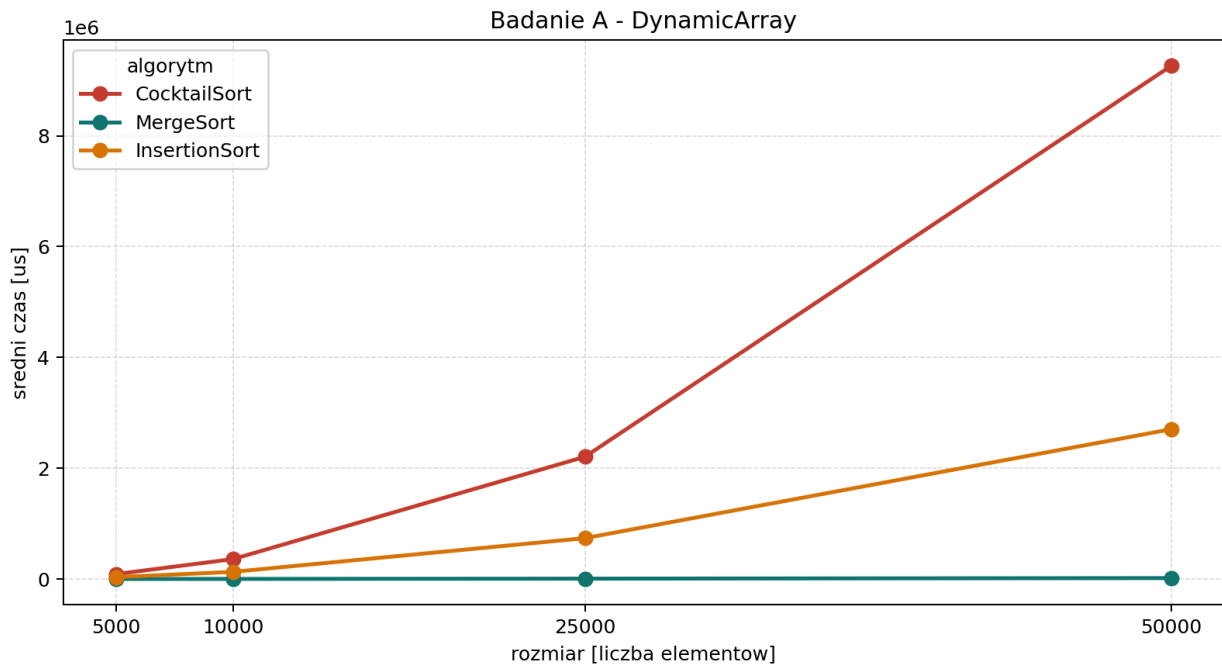
`InsertionSort` przetwarza dane od lewej do prawej. W każdej iteracji pobiera bieżący element, porównuje go z elementami poprzedzającymi i przesuwa większe wartości o jedno miejsce w prawo, aż znajdzie pozycję, w którą można wstawić rozpatrywany element. Dla `DynamicArray` operacja ta jest realizowana bezpośrednio na indeksach tablicy [1][5].

Dla `SinglyLinkedList` zastosowano drugą wersję algorytmu. Zamiast próbować przesuwać elementy przez kosztowny dostęp indeksowy, implementacja buduje nową uporządkowaną część listy. Kolejne węzły są wyjmowane z listy wejściowej i wstawiane w odpowiednie miejsce już posortowanej części. Dzięki temu algorytm korzysta z naturalnych operacji listowych i nie opiera się na `operator[]` [2][5].

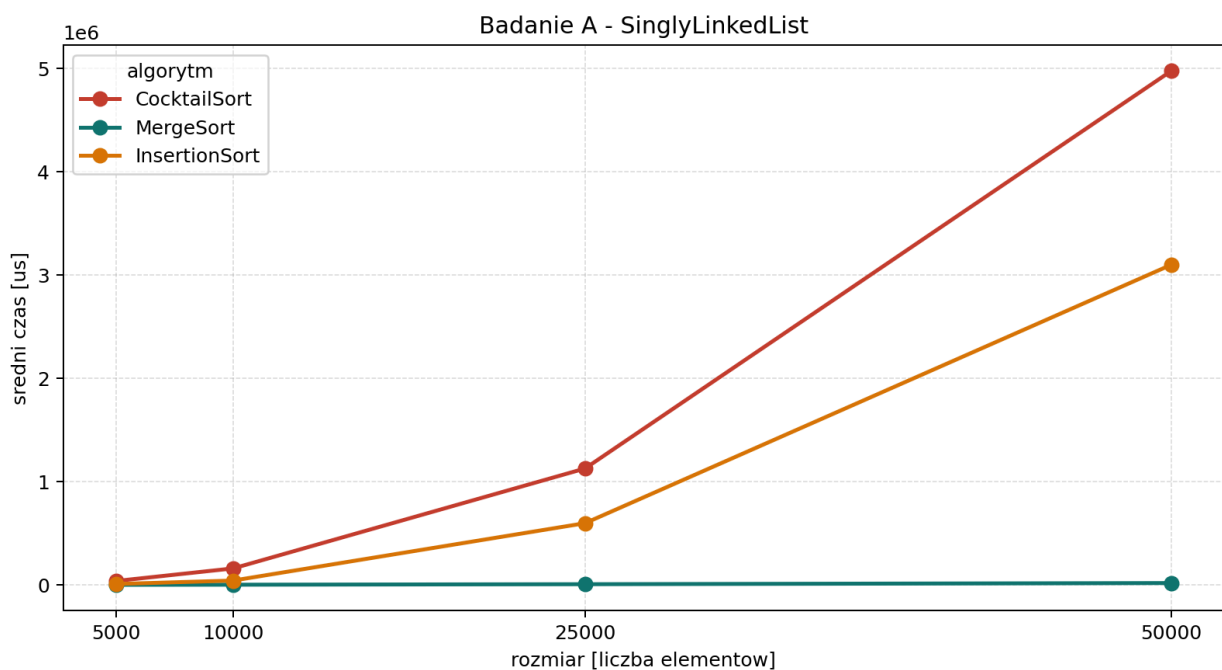
4. Wyniki badań

4.1. Badanie A

Badanie A sprawdza wpływ liczebności zbioru na czas działania algorytmów. W tej części wykonano 24 benchmarki, czyli wszystkie kombinacje trzech algorytmów, dwóch struktur i czterech rozmiarów wejścia. Już na poziomie wykresów widać, że **MergeSort** skaluje się znacznie lepiej niż **CocktailSort** i **InsertionSort**, zwłaszcza dla większych rozmiarów danych, co jest zgodne z klasyczną analizą złożoności [1][4][5].



Rysunek 1: Badanie A dla struktury DynamicArray.



Rysunek 2: Badanie A dla struktury SinglyLinkedList.

Algorytm	Rozmiar	Min [us]	Avg [us]	Max [us]
CocktailSort	5000	83195	89108.56	152162
CocktailSort	10000	339583	360762.9	482766
CocktailSort	25000	2113922	2207216.92	2461371
CocktailSort	50000	8560470	9263002.6	11764091
MergeSort	5000	1065	1418.6	5114
MergeSort	10000	2306	2792.2	6513
MergeSort	25000	6366	7649.94	10991
MergeSort	50000	13195	17923.3	41499
InsertionSort	5000	32608	34486.62	39518
InsertionSort	10000	121210	130260.08	157428
InsertionSort	25000	647881	738587	856527
InsertionSort	50000	2576810	2703807.76	2915133

Tabela 1: Czas sortowania w zależności od liczebności zbioru dla struktury `DynamicArray`.

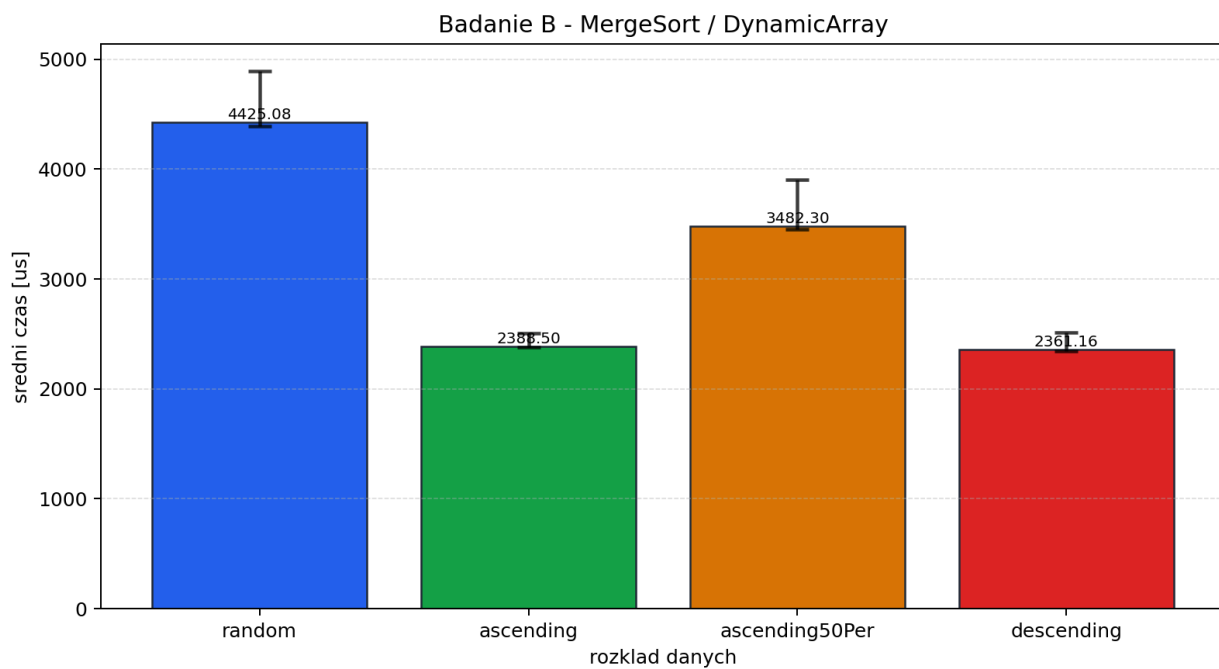
Algorytm	Rozmiar	Min [us]	Avg [us]	Max [us]
CocktailSort	5000	36606	38083.34	41571
CocktailSort	10000	156028	159970.48	167807
CocktailSort	25000	1082248	1126753.44	1421759
CocktailSort	50000	4437247	4975341.42	7575448
MergeSort	5000	740	967.32	3414
MergeSort	10000	1639	2139.36	3088
MergeSort	25000	5038	7023.76	20209
MergeSort	50000	9133	18292.06	100426
InsertionSort	5000	8813	9024.06	9814
InsertionSort	10000	39704	42301.42	96177
InsertionSort	25000	564377	597466.82	649092
InsertionSort	50000	2980966	3098944.34	3684913

Tabela 2: Czas sortowania w zależności od liczebności zbioru dla struktury `SinglyLinkedList`.

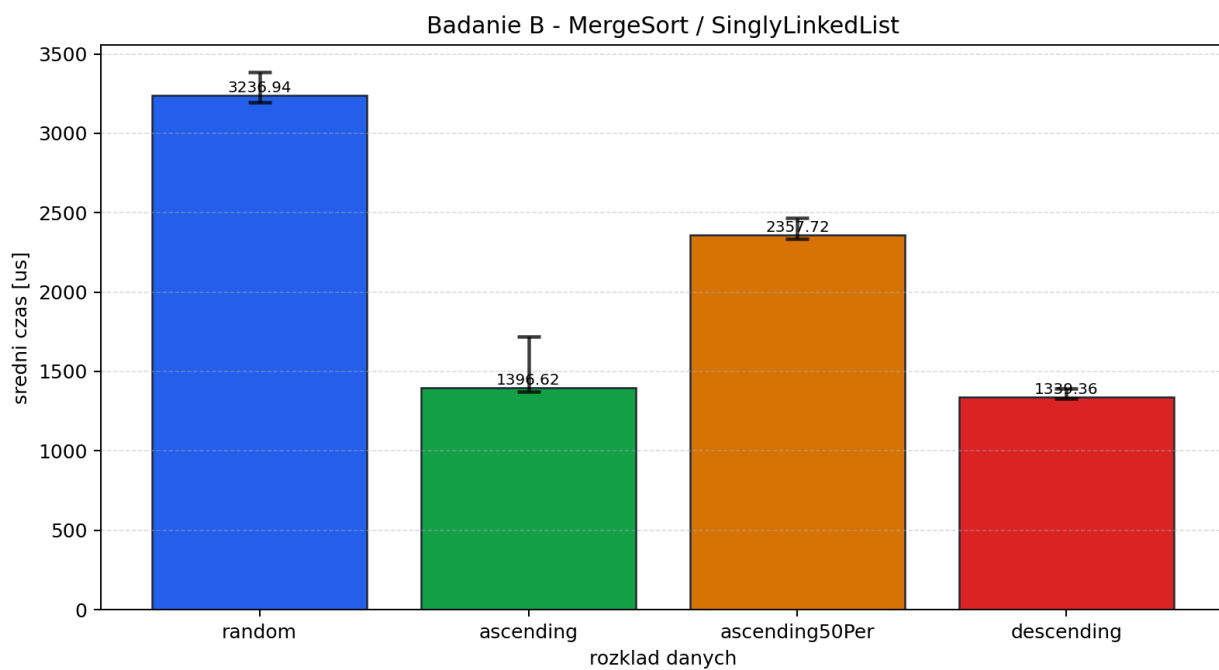
Przykładowo dla `DynamicArray` i rozmiaru 50000 średni czas `MergeSort` wyniósł 17923.3 us, podczas gdy `CocktailSort` osiągnął 9263002.6 us, a `InsertionSort` 2703807.76 us. Oznacza to bardzo dużą różnicę pomiędzy algorytmem $O(n \log n)$ a algorytmami o dominującym zachowaniu $O(n^2)$ [1][4][5].

4.2. Badanie B

Badanie B dotyczy wpływu rozkładu danych wejściowych. W tej części wykonano 8 benchmarków: po cztery dla `DynamicArray` i `SinglyLinkedList`. Każdy benchmark używał `MergeSort`, a zmieniano jedynie rozkład danych wejściowych. Wykresy i tabela pokazują, że dla tego algorytmu rozkład wejścia wpływa na wynik dużo słabiej niż sama liczebność zbioru w badaniu A.



Rysunek 3: Badanie B dla MergeSort i struktury DynamicArray.



Rysunek 4: Badanie B dla MergeSort i struktury SinglyLinkedList.

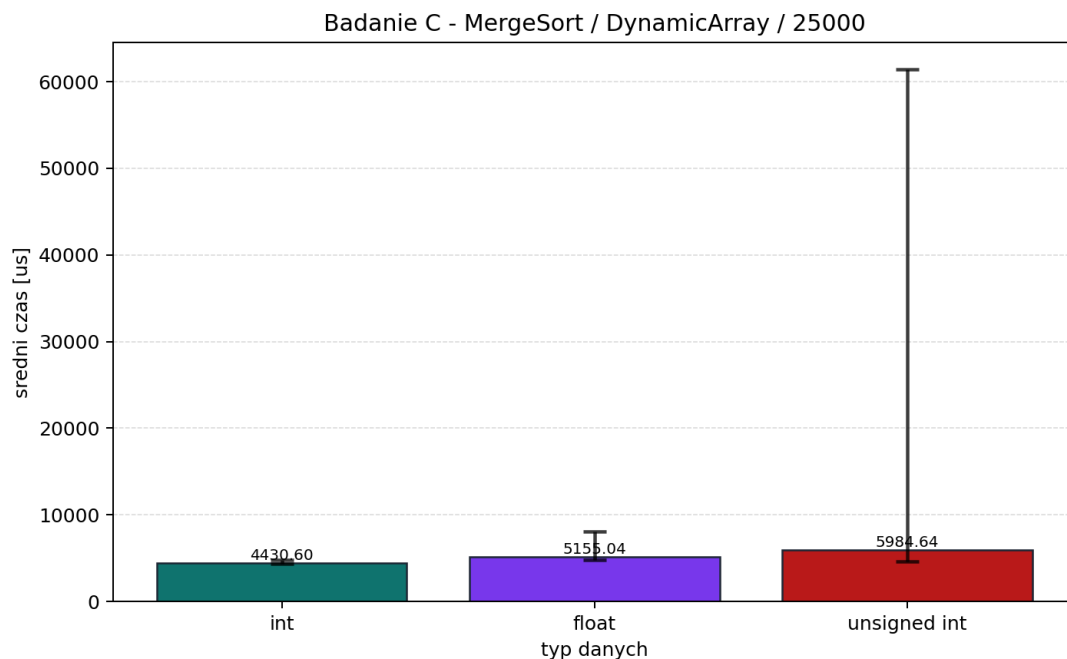
Struktura	Rozkład danych	Min [us]	Avg [us]	Max [us]
DynamicArray	random	4391	4425.08	4894
DynamicArray	ascending	2379	2388.5	2506
DynamicArray	ascending50Per	3455	3482.3	3906
DynamicArray	descending	2345	2361.16	2511
SinglyLinkedList	random	3194	3236.94	3386
SinglyLinkedList	ascending	1372	1396.62	1722
SinglyLinkedList	ascending50Per	2335	2357.72	2467
SinglyLinkedList	descending	1331	1339.36	1391

Tabela 3: Czas sortowania w zależności od początkowego rozkładu elementów.

Dla `DynamicArray` średni czas dla rozkładu `random` wyniósł 4425.08 us, dla `ascending` 2388.5 us, dla `ascending50Per` 3482.3 us, a dla `descending` 2361.16 us. Analogiczny trend widać dla `SinglyLinkedList`, choć poziom czasów jest tam inny ze względu na inną organizację danych.

4.3. Badanie C

Badanie C analizuje wpływ typu danych przy zachowaniu tego samego algorytmu, tej samej struktury oraz tego samego rozmiaru wejścia. W tej części wykonano 3 benchmarki: dla `int`, `float` i `unsigned int`. Wyniki pokazują, że w tej konfiguracji dominujący wpływ ma sam algorytm i organizacja danych, a nie drobne różnice pomiędzy badanymi typami [1][5].



Rysunek 5: Badanie C dla MergeSort, DynamicArray i rozmiaru 25000.

Typ danych	Min [us]	Avg [us]	Max [us]
int	4382	4430.6	4782
float	4751	5155.04	8046
unsigned int	4579	5984.64	61441

Tabela 4: Czas sortowania w zależności od typu danych.

Średnie czasy wyniosły odpowiednio:

- `int`: 4430.6 us
- `float`: 5155.04 us
- `unsigned int`: 5984.64 us

5. Analiza wyników

W analizie wyników warto rozdzielić dwie grupy algorytmów. `CocktailSort` oraz `InsertionSort` należą tutaj do algorytmów o dominującym zachowaniu kwadratowym $O(n^2)$, natomiast `MergeSort` zachowuje złożoność $O(n \log n)$ [1][4][5]. Ta różnica jest widoczna we wszystkich trzech głównych badaniach, ale każde z nich pokazuje ją z innej strony.

5.1. Badanie A

Badanie A pokazuje przede wszystkim wpływ liczebności zbioru. Wraz ze wzrostem rozmiaru wejścia czasy `CocktailSort` i `InsertionSort` rosną znacznie szybciej niż czasy `MergeSort`. Najmocniej widać to przy 50000 elementów, gdzie `MergeSort` pozostaje wyraźnie szybszy zarówno dla `DynamicArray`, jak i dla `SinglyLinkedList`.

Na poziomie implementacji taki wynik jest w pełni uzasadniony. `MergeSort` dla `DynamicArray` wykonuje podział zakresu i scalanie uporządkowanych fragmentów z użyciem jednego dodatkowego bufora, czyli tymczasowej tablicy pomocniczej. Dla `SinglyLinkedList` ten sam algorytm nie wykonuje kosztownego dostępu indeksowego, lecz dzieli listę na połowy i scala ją przez przepinanie wskaźników `next`. Dzięki temu obie wersje zachowują się zgodnie z oczekiwaną teorią [1][2][5].

`CocktailSort` oraz `InsertionSort` są znacznie bardziej wrażliwe na wzrost liczby elementów. W przypadku `SinglyLinkedList` autor implementacji musiał dodatkowo uważać na koszt dostępu do elementu pod indeksem. Z tego powodu `CocktailSort` otrzymał osobną wersję wykorzystującą tablicę wskaźników do węzłów, a `InsertionSort` osobną wersję budującą uporządkowaną część listy przez wstawianie węzłów we właściwe miejsce. Bez takich zmian wykorzystanie samego `operator[]` prowadziłoby do bardzo dużego dodatkowego narzutów [2][5].

5.2. Badanie B

W badaniu B rozkład danych nie zmienia jakościowo zachowania `MergeSort`, ale wpływa na konkretne czasy wykonania. W szczególności przypadek `ascending50Per` bywa wolniejszy od `descending`, mimo że oba należą do tego samego głównego badania. Wynika to z tego, że dane `ascending50Per` nie są całkowicie uporządkowane: połowa wejścia jest posortowana, a reszta pozostaje losowa. Przy scalaniu takich fragmentów algorytm wykonuje więcej nieregularnych porównań niż w bardziej regularnym przypadku całkowicie malejącym. Wpływ mają tu również szczegóły implementacyjne i lokalność pamięci [1][5].

5.3. Badanie C

Badanie C pokazuje wpływ typu danych przy stałym algorytmie, stałej strukturze i stałym rozmiarze wejścia. Dla `MergeSort` i `DynamicArray` średnie czasy dla `int`, `float` i `unsigned int` różnią się, ale te różnice są dużo mniejsze niż te, które w badaniu A wynikały ze zmiany algorytmu. Innymi słowy, przy tej konfiguracji ważniejsze jest to, że użyto `MergeSort`, niż to, czy sortowany był `int`, `float` czy `unsigned int`.

Warto też zauważyć, że największy średni czas w badaniu C uzyskał `unsigned int`. Nie oznacza to jednak zmiany klasy złożoności algorytmu. Jest to raczej efekt szczegółów wykonania i konkretnego zestawu danych wylosowanych do benchmarku. Główna obserwacja pozostaje taka sama: wpływ typu danych istnieje, ale jest słabszy niż wpływ wyboru algorytmu i sposobu organizacji struktury [1][4][5].

6. Wnioski

Najważniejszy wniosek z przeprowadzonych eksperymentów jest taki, że **MergeSort** wyraźnie domi-
nuje przy większych rozmiarach danych. Już w badaniu A widać, że przy 50000 elementów różnica
pomiędzy **MergeSort** a pozostałymi algorytmami nie jest kosmetyczna, lecz bardzo duża. Oznacza
to, że przy większych wejściach przewaga złożoności $O(n \log n)$ przekłada się bezpośrednio na
praktyczny czas działania.

Drugi istotny wniosek dotyczy samej struktury danych i sposobu implementacji. **SinglyLinkedList**
wymagała osobnych wersji części algorytmów, ponieważ prosty dostęp indeksowy nie odpowiada
naturze listy jednokierunkowej. Wyniki pokazują więc nie tylko różnice między teorią poszczegól-
nych algorytmów, lecz także to, że dobór sposobu implementacji do właściwości struktury danych
ma realny wpływ na końcowy rezultat.

Trzeci wniosek płynący z badań B i C jest bardziej praktyczny. Zmiana rozkładu wejścia lub
prostego typu liczbowego wpływa na wyniki, ale wpływ ten jest słabszy niż zmiana samego
algorytmu. W badanym zakresie to właśnie wybór algorytmu i dopasowanie go do struktury danych
okazały się najważniejsze dla osiągnięcia dobrych czasów sortowania.

7. Literatura

- [1] Maciej M. Sysło, *Algorytmy*, Helion, 2016.
- [2] L. Banachowski, K. Diks, W. Rytter, *Algorytmy i struktury danych*, Wydawnictwo Naukowe PWN, Warszawa, 2018.
- [3] R. Lazarus, R. Frank, *A High-Speed Sorting Procedure*, Communications of the ACM, 3(1), 1960.
- [4] Marcin Kasprowicz, *Złożoność obliczeniowa algorytmu*, algorytm.edu.pl, dostęp online.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Wprowadzenie do algorytmów*, PWN, 2022.